Proiect Simulare Urbana 3D

Documentatie Tehnica

Autor:
Campean Bogdan

Universitatea Tehnica din Cluj-Napoca
Facultatea de Automatica si Calculatoare

January 10, 2026

Contents

1 Prezentarea temei **3**

2 Scenariul **3**
2.1 Descrierea scenei si a obiectelor . . . 3
2.2 Functionalitati . . . 3

3 Detalii de implementare **4**
3.1 Implementarea Randarii si Transformarilor Geometrice . . . 4
3.1.1 Functia RenderModel . . . 4
3.1.2 Randarea Mediului (Sol si Skybox) . . . 5
3.2 Sistemul de Iluminare . . . 5
3.2.1 Structuri de date si Uniforme . . . 5
3.2.2 Calculul si trimiterea luminilor in scena . . . 6
3.3 Generarea Umbrelor (Shadow Mapping) . . . 6
3.4 Randarea Vehiculului (RenderCar) . . . 7
3.5 Sistemul de Particule . . . 8
3.5.1 Structura de Date . . . 8
3.5.2 Logica de Generare si Actualizare (Fizica) . . . 8
3.5.3 Randarea si Tehnica de Billboarding . . . 9
3.6 Fizica, Coliziuni si Debugging . . . 10
3.6.1 Algoritmul AABB (Axis-Aligned Bounding Box) . . . 10
3.6.2 Verificarea Predictiva a Coliziunilor . . . 10
3.6.3 Sistemul de Vizualizare Debug (X-Ray) . . . 11
3.6.4 Coliziunea Dinamica a Usii de Garaj . . . 11
3.7 Simularea Fizica a Vehiculului . . . 13
3.7.1 Modelul de Acceleratie si Inertie . . . 13
3.7.2 Calculul Traiectoriei si Directia . . . 13
3.7.3 Cinematica Rotilor (Spin si Viraj) . . . 14

4 Manual de utilizare **14**
4.1 Control General . . . 14
4.2 Taste Functionale . . . 15
4.3 Moduri de Vizualizare (Debug) . . . 15

5 Concluzii si dezvoltari ulterioare **15**

6 Referinte **15**

1 Prezentarea temei

Proiectul consta in realizarea unei simulari 3D a unui mediu urban (cartier rezidential), avand ca obiectiv principal implementarea si demonstrarea unor concepte fundamentale si avansate de grafica pe calculator utilizand libraria OpenGL.

Tema propusa este o "Simulare Urbana Interactiva", in care utilizatorul poate explora scena atat din perspectiva "free-cam", cat si din perspectiva unui sofer, controland un vehicul cu fizica simulata (inertie, acceleratie). Proiectul pune accent pe realism prin utilizarea iluminarii dinamice, a umbrelor generate in timp real (Shadow Mapping), a efectelor de particule (fum) si a detectiei coliziunilor.

2 Scenariul

2.1 Descrierea scenei si a obiectelor

Scena este construita sub forma unui cartier rezidential detaliat. Elementele principale care compun mediul virtual sunt:

- **Vehiculul:** O masina detaliata, compusa din caroserie si roti separate, capabile sa se roteasca si sa vireze in concordanta cu comenzile utilizatorului. Masina este echipata cu faruri functionale (spotlights) si emite particule de fum, proportionale cu acceleratia(adica cat timp este apsata tasta s/w scoate mai mult fum).
- **Garajul Interactiv:** O structura de tip garaj care contine o usa animata(obiect separat de garajul principal). Usa se poate deschide si inchide la comanda, iar sistemul de coliziuni se actualizeaza dinamic (masina poate intra doar cand usa este ridicata).
- **Cladiri si Mediu:** Scena contine o casa principala, blocuri de apartamente si elemente de infrastructura urbana (drumuri, trotuare, lampi stradale, banci, garduri).
- **Vegetatie Generata Procedural:** Pentru a popula zona verde din interiorul gardului(un mini parc), am implementat un algoritm care plaseaza aleatoriu copaci intr-o zona definita.
- **Skybox:** Un cub texturat care simuleaza cerul si orizontul, oferind profunzime scenei.Pozitia luminii directionale din scena este directia soarelui de pe Skybox.
- **Fenomene meteo:** Implementarea cetii (fog) direct in Fragment Shader, folosind o functie exponentiala pentru a atenua marginile scenei si a adauga realism.

2.2 Functionalitati

Utilizatorul are control total asupra simularii prin intermediul tastaturii si al mouse-ului. Principalele functionalitati includ:

- Comutarea intre modul "Condus" (camera urmareste masina cand se deplaseaza) si modul "free-cam" (camera se deplaseaza libera de masina,care ramane pe loc).
- Controlul fizic al masinii cu inertie si detectia coliziunilor cu obstacolele statice.

- Actionarea mecanismelor din scena (deschiderea/inchiderea usii de garaj).
- Vizualizarea cutiilor de incadrare (Bounding Boxes).
- Moduri de randare diferite pentru debug (Wireframe, Poligonal).
- Buton pentru prezentarea scenei.

3 Detalii de implementare

Aceasta sectiune detaliaza arhitectura software a proiectului, explicand algoritmii utilizati pentru randare, iluminare, fizica si efecte speciale, insotiti de fragmente relevante din codul sursa.

3.1 Implementarea Randarii si Transformarilor Geometrice

Pentru a gestiona eficient desenarea numarului mare de obiecte statice din scena, am abstractizat procesul de randare in doua functii principale: `RenderModel` si `RenderModelRotated`. Aceasta abordare modulara permite generarea scenei printr-o singura linie de cod per obiect.[2]

3.1.1 Functia RenderModel

Aceasta este functia de baza utilizata pentru majoritatea obiectelor. Ea construieste Matricea de Model (Model Matrix) aplicand transformari in ordinea standard: Scalare → Rotatie → Translatie.

```
void RenderModel(gps::Model3D& modelObj, gps::Shader shader, glm
    ::vec3 position, glm::vec3 scale, float rotationAngle, glm::
    vec3 rotate) {
    shader.useShaderProgram();

    // Initializare matrice identitate
    glm::mat4 model = glm::mat4(1.0f);

    // Constructia matricii Model (Translatie -> Rotatie ->
        Scalare)
    model = glm::translate(model, position);
    model = glm::rotate(model, glm::radians(rotationAngle),
        rotate);
    model = glm::scale(model, scale);

    // Trimiterea datelor la Shader
    glUniformMatrix4fv(glGetUniformLocation(shader.shaderProgram,
         "model"), 1, GL_FALSE, glm::value_ptr(model));

    // Calculul si trimiterea Matricii Normale (esentiala pentru
        iluminare corecta)
    // Se foloseste inversa transpusei pentru a pastra
        perpendicularitatea normalelor
```

```
    glm::mat3 normalMatrix = glm::mat3(glm::inverseTranspose(view * model));
    glUniformMatrix3fv(glGetUniformLocation(shader.shaderProgram, "normalMatrix"), 1, GL_FALSE, glm::value_ptr(normalMatrix));

    modelObj.Draw(shader);
}
```

3.1.2 Randarea Mediului (Sol si Skybox)

Elementele ambientale sunt tratate separat pentru performanta si corectitudine vizuala:

Solul (initFloor/renderFloor): Nu este un model incarcat, ci o geometrie generata manual (doua triunghiuri mari). Textura este setata pe modul `GL_REPEAT`.

```
// Generarea geometriei solului (Quad)
GLfloat floorVertices[] = {
    -1000.0f, 0.0f,  1000.0f,  0.0f, 1.0f, 0.0f,  0.0f, 0.0f,
     1000.0f, 0.0f,  1000.0f,  0.0f, 1.0f, 0.0f,  1000.0f, 0.0f,
     ... // Alte varfuri
};
```

Skybox (initSkyBox): Foloseste o textura de tip `CubeMap`. La randare, componenta de translatie a matricii de vizualizare (`view`) este eliminata pentru a crea efectul de infinit.

```
// Randarea Skybox-ului
skyboxShader.useShaderProgram();
glDisable(GL_CULL_FACE); // Vedem interiorul cubului
mySkyBox.Draw(skyboxShader, view, projection);
```

3.2 Sistemul de Iluminare

Shaderul principal (`basic.frag`) implementeaza un model de iluminare Blinn-Phong care insumeaza contributia a trei tipuri de surse de lumina.[2]

3.2.1 Structuri de date si Uniforme

Pentru a gestiona sursele de lumina complexe, in special farurile masinii, am definit structuri in GLSL si vectori corespunzatori in C++.

Structura SpotLight (pentru faruri): Aceasta structura permite crearea unor conuri de lumina cu margini difuze (soft edges).

```
// Definitie in Fragment Shader
struct SpotLight {
    vec3 position;
    vec3 direction;
    float cutOff;       // Unghiul cercului interior (lumina maxima)
    float outerCutOff;  // Unghiul cercului exterior (fade out)
    float constant;
    float linear;
```

```
    float quadratic;    // Coeficienti pentru atenuare cu distanta
    vec3 color;
};
```

3.2.2 Calculul si trimiterea luminilor in scena

In functia `renderScene`, luminile sunt actualizate la fiecare cadru:

Lumini Pozitionale (Point Lights): Scena contine 6 lumini pozitionale(4 pe cladirile tip blocuri 1 pe casa 1 pe felinar).

```
// Trimitere array de lumini punctiforme
for (unsigned int i = 0; i < 6; i++) {
    string number = to_string(i);
    glUniform3fv(glGetUniformLocation(shader, ("lightPos[" + number + "]").c_str()), 1, value_ptr(pointLightPositions[i]));
    glUniform3fv(glGetUniformLocation(shader, ("lightColorPoint[" + number + "]").c_str()), 1, value_ptr(pointLightColors[i]));
}
```

Lumini Spot (Faruri Dinamice): Pozitia si directia farurilor sunt calculate in timp real, folosind matricea de rotatie a masinii. [3]

```
// Calcul pozitie faruri (offset rotit)
glm::mat4 carRotationMatrix = glm::rotate(glm::mat4(1.0f), glm::radians(carAngle), glm::vec3(0.0f, 1.0f, 0.0f));
glm::vec3 rotatedLeft = glm::vec3(carRotationMatrix * glm::vec4(headlightOffsetLeft, 1.0f));
glm::vec3 finalPosLeft = carPosition + rotatedLeft;

// Trimitere catre shader
glUniform3fv(glGetUniformLocation(shader, "spotLight[0].position"), 1, value_ptr(finalPosLeft));
glUniform3fv(glGetUniformLocation(shader, "spotLight[0].direction"), 1, value_ptr(carDirection));
```

3.3 Generarea Umbrelor (Shadow Mapping)

Umbrele sunt generate folosind un algoritm in doi pasi (Two-Pass Rendering) implementat in functia `renderScene`.[2]

Pasul 1: Randarea hartii de adancime (Depth Pass) Scena este randata din perspectiva sursei de lumina (Soarele) intr-un Framebuffer personalizat.

```
// Configurare matrice lumina (Ortografica)
glm::mat4 lightProjection = glm::ortho(-60.0f, 60.0f, -60.0f, 60.0f, near_plane, far_plane);
glm::mat4 lightView = glm::lookAt(currentLightPos, shadowCenter, glm::vec3(0.0f, 1.0f, 0.0f));
glm::mat4 lightSpaceTrMatrix = lightProjection * lightView;
```

```

// Randare in Depth Map FBO
glBindFramebuffer(GL_FRAMEBUFFER, shadowMapFBO);
glClear(GL_DEPTH_BUFFER_BIT);
drawObjects(depthMapShader, true);
```

Pasul 2: Randarea finala (Render Pass) Scena este randata normal. Matricea de transformare a luminii si textura de adancime sunt trimise la shader pentru comparatie.

```
// Trimiterea matricii de lumina si a texturii de umbra
glUniformMatrix4fv(glGetUniformLocation(shader, "lightSpaceTrMatrix"), 1, GL_FALSE, value_ptr(lightSpaceTrMatrix));
glActiveTexture(GL_TEXTURE3);
glBindTexture(GL_TEXTURE_2D, depthMapTexture);
glUniform1i(glGetUniformLocation(shader, "shadowMap"), 3);
```

3.4 Randarea Vehiculului (RenderCar)

Functia `renderCar` gestioneaza logica vizuala a masinii. Vehiculul este un ansamblu compus din caroserie si 4 roti independente.

```
void renderCar(gps::Shader shader) {
    // 1. Randare Caroserie
    glm::mat4 modelCar = glm::translate(glm::mat4(1.0f), carPosition);
    modelCar = glm::rotate(modelCar, glm::radians(carAngle), glm::vec3(0.0f, 1.0f, 0.0f));
    // ... Alte transformari si desenare ...

    // 2. Randare Roti (Iterativ)
    for (int i = 0; i < 4; i++) {
        glm::mat4 modelWheel = glm::mat4(1.0f);
        // Ierarhie de transformari:
        modelWheel = glm::translate(modelWheel, carPosition); // A. Pozitia Masinii
        modelWheel = glm::rotate(modelWheel, glm::radians(carAngle), glm::vec3(0, 1, 0)); // B. Orientarea Masinii
        modelWheel = glm::translate(modelWheel, wheelOffsets[i]); // C. Offset roata

        // D. Viraj (Doar fata)
        if (i < 2) modelWheel = glm::rotate(modelWheel, glm::radians(steeringAngle), glm::vec3(0, 1, 0));

        // E. Invartire (Spin)
        modelWheel = glm::rotate(modelWheel, glm::radians(wheelAngle), glm::vec3(1, 0, 0));

        // ... Desenare ...
    }
```

```
}
```

3.5 Sistemul de Particule

Pentru a simula gazele de esapament emise de autovehicul, am implementat un sistem de particule gestionat pe CPU (Central Processing Unit). Aceasta abordare ofera un control fin asupra logicii de generare si fizicii particulelor, fiind ideala pentru un numar moderat de elemente (sub 10.000).

Sistemul functioneaza pe baza unui ciclu continuu: **Generare** → **Actualizare** → **Stergere** → **Randare**. [3]

3.5.1 Structura de Date

Fiecare particula este o instanta a unei structuri C++ care retine starea curenta a acesteia. Folosim un container `std::vector` pentru a gestiona lista dinamica a particulelor active.

```
// Definirea unei particule individuale
struct Particle {
    glm::vec3 position; // Pozitia in spatiul 3D
    glm::vec3 velocity; // Vectorul de directie si viteza
    float life;         // Durata de viata ramasa (1.0 -> 0.0)
    float size;         // Dimensiunea curenta (fumul se extinde)
};

// Containerul principal
std::vector<Particle> smokeParticles;
```

3.5.2 Logica de Generare si Actualizare (Fizica)

Functia `updateSmokeParticles` este apelata la fiecare cadru si indeplineste doua roluri majore:

1. Generarea (Emisia): Emisia este conditionata de un timer. Frecventa si forta emisiei depind de starea masinii (input-ul utilizatorului). Daca tasta `W` este apasata (acceleratie), intervalul de generare scade (fum mai des), iar viteza de ejectie creste.

```
// Determinarea parametrilor de emisie
float spawnInterval = 0.2f; // Idle (motor la relanti)
float ejectionForce = 2.0f;

if (pressedKeys[GLFW_KEY_W]) {
    spawnInterval = 0.04f;  // Accelerare (de 5x mai des)
    ejectionForce = 6.0f;   // Presiune mare
}

// Crearea unei noi particule
if (smokeSpawnTimer >= spawnInterval) {
    Particle p;
    // Pozitia este fixata la teava de esapament, rotita cu
       sasiul
    p.position = carPosition + rotatedExhaustOffset;
```

```

    // Viteza este in spatele masinii + o componenta random
    p.velocity = backwardDir * ejectionForce + randomVector;
    p.life = 1.0f;
    smokeParticles.push_back(p);
}
```

2. Actualizarea (Update Loop): Iteram prin vectorul de particule si aplicam legile fizice elementare (Euler Integration).

- **Pozitie:** $Pozitie = Pozitie + Viteza \times \Delta t$
- **Decelerare:** Viteza scade in timp (frecarea cu aerul).
- **Expansiune:** Dimensiunea (`size`) creste treptat, simuland dispersia gazului.
- **Imbatranire:** Variabila `life` scade. Cand atinge 0, particula este stearsa din memorie pentru a mentine performanta.

3.5.3 Randarea si Tehnica de Billboarding

Vizual, fiecare particula este un simplu patrat (Quad) texturat. Provocarea in 3D este ca acest patrat sa para o sfera de fum, indiferent de unghiul din care privim.

Am implementat tehnica de **Spherical Billboarding**. Aceasta asigura ca suprafata patratului este intotdeauna perpendiculara pe vectorul de privire al camerei.

In loc sa calculam rotatii complexe, folosim o proprietate a matricilor: Matricea de Vizualizare (`view`) contine vectorii de orientare ai camerei. Pentru a alinia particula cu camera, copiem acesti vectori in Matricea de Model a particulei.

```
void renderSmoke() {
    particleShader.useShaderProgram();
    glEnable(GL_BLEND); // Activam transparenta

    for (const auto& p : smokeParticles) {
        glm::mat4 model = glm::mat4(1.0f);
        model = glm::translate(model, p.position);

        // --- BILLBOARDING MAGIG ---
        // Anulam rotatia obiectului setand coloanele matricii
           Model
        // cu vectorii de baza ai camerei (View Matrix)
        // Astfel, particula "priveste" mereu spre ecran.
        model[0][0] = view[0][0]; model[0][1] = view[1][0]; model
           [0][2] = view[2][0];
        model[1][0] = view[0][1]; model[1][1] = view[1][1]; model
           [1][2] = view[2][1];
        model[2][0] = view[0][2]; model[2][1] = view[1][2]; model
           [2][2] = view[2][2];

        model = glm::scale(model, glm::vec3(p.size));

        // Trimitem opacitatea (life) la shader pentru Fade Out
```

```
        glUniform1f(glGetUniformLocation(shader, "opacity"), p.
            life);
        glUniformMatrix4fv(..., value_ptr(model));

        glDrawArrays(GL_TRIANGLES, 0, 6);
    }
}
```

Aceasta tehnica este extrem de eficienta si creeaza iluzia unor nori volumetrici 3D folosind doar geometrie 2D.

3.6 Fizica, Coliziuni si Debugging

Un aspect fundamental al proiectului este interactiunea fizica dintre vehicul si mediul inconjurator. Pentru a preveni trecerea masinii prin cladiri sau iesirea acesteia din limitele hartii, am implementat un sistem robust de detectie a coliziunilor bazat pe volume de incadrare.

3.6.1 Algoritmul AABB (Axis-Aligned Bounding Box)

Fiecare obiect static (cladire, copac, gard) este aproximat printr-un paralelipiped dreptunghic aliniat la axele de coordonate (X, Y, Z). Aceasta simplificare reduce drastic complexitatea calculelor, transformand testul de intersectie in simple comparatii de coordonate.[3]

Structura de baza este definita astfel:

```
struct BoundingBox {
    glm::vec3 min; // Punctul stanga-jos-spate (coordonate minime
        )
    glm::vec3 max; // Punctul dreapta-sus-fata (coordonate maxime
        )
};
```

Functia de detectie verifica daca exista suprapunere pe toate cele trei axe simultan. Daca intervalele [min, max] ale celor doua cutii se intersecteaza pe X, Y si Z, atunci obiectele sunt in coliziune.

```
bool checkCollision(BoundingBox a, BoundingBox b) {
    // Verificam suprapunerea pe axa X
    bool collisionX = a.min.x <= b.max.x && a.max.x >= b.min.x;
    // Verificam suprapunerea pe axa Y
    bool collisionY = a.min.y <= b.max.y && a.max.y >= b.min.y;
    // Verificam suprapunerea pe axa Z
    bool collisionZ = a.min.z <= b.max.z && a.max.z >= b.min.z;

    return collisionX && collisionY && collisionZ;
}
```

3.6.2 Verificarea Predictiva a Coliziunilor

Pentru a evita situatia in care masina ramane blocata *in interiorul* unui zid, verificarea coliziunii se face **predictiv**. La fiecare cadru, calculam pozitia viitoare (`nextPos`) unde

ar ajunge masina daca s-ar aplica viteza curenta. Construim bounding box-ul masinii in acea pozitie viitoare si il testam impotriva tuturor obstacolelor.

Daca `checkCollision` returneaza `true`, miscarea este anulata, iar pozitia masinii (`carPosition`) ramane neschimbata, simuland un impact solid.

3.6.3 Sistemul de Vizualizare Debug (X-Ray)

Pentru a verifica corectitudinea dimensiunilor cutiilor de coliziune, am implementat un mod de vizualizare dedicat, activat prin tasta **B**.

Acest mod randeaza "scheletul" coliziunilor folosind urmatoarele tehnici OpenGL:

- **Wireframe Mode:** `glPolygonMode(..., GL_LINE)` deseneaza doar muchiile poligoanelor.
- **Depth Test Disable:** `glDisable(GL_DEPTH_TEST)` dezactiveaza verificarea adancimii. Astfel, liniile rosii ale cutiilor sunt desenate peste orice alt obiect din scena, creand un efect de "X-Ray" (vedere prin pereti). Acest lucru este crucial pentru a vedea cutia masinii chiar daca aceasta este fizic in interiorul caroseriei.

```
if (showBoundingBoxes) {
    myBasicShader.useShaderProgram();
    glDisable(GL_DEPTH_TEST); // Efect X-Ray
    glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);

    // 1. Randare obstacole statice
    for (const auto& box : obstacles) {
        drawDebugBox(box, myBasicShader);
    }

    // 2. Randare cutie dinamica masina
    // Cutia trebuie sa se roteasca si sa se miste odata cu
       masina
    glm::mat4 modelCar = glm::mat4(1.0f);
    modelCar = glm::translate(modelCar, carPosition);
    modelCar = glm::rotate(modelCar, glm::radians(carAngle), glm
       ::vec3(0, 1, 0));
    modelCar = glm::scale(modelCar, glm::vec3(2.5f, 2.8f, 4.0f));
        // Dimensiuni exacte
    // Centrare cub unitar (-0.5 la 0.5)
    modelCar = glm::translate(modelCar, glm::vec3(-0.5f, 0.0f,
       -0.5f));

    glUniformMatrix4fv(..., value_ptr(modelCar));
    glDrawArrays(GL_LINES, 0, 24);
}
```

3.6.4 Coliziunea Dinamica a Usii de Garaj

Usa de garaj reprezinta un obstacol dinamic, spre deosebire de cladiri sau copaci. Gestionarea acesteia necesita o abordare hibrida care combina transformari geometrice cu logica de stare conditionala.

1. Matematica Animatiei (Interpolare) Miscarea usii nu este o simpla rotatie, ci o combinatie complexa de translatie pe verticala (axa Y) si retragere pe orizontala (axa Z), sincronizata cu unghiul de deschidere. Am definit o variabila `normalizedTime` in intervalul $[0, 1]$ derivata din unghiul curent al usii ($0° \rightarrow 90°$).

Pozitia bounding box-ului la orice moment t este calculata astfel:

$$P_{animata} = P_{baza} + \vec{V}_{up} \cdot t - \vec{V}_{out} \cdot t$$

Unde $\vec{V}_{up}$ este vectorul de ridicare (4.2 unitati) si $\vec{V}_{out}$ este vectorul de iesire in consola (2.5 unitati).

```
// Calculul pozitiei in timp real
float normalizedTime = garageDoorAngle / 90.0f;
float slideUp = normalizedTime * 4.2f;
float slideOut = normalizedTime * 2.5f;

// Noua pozitie a centrului usii
glm::vec3 animPos = basePos;
animPos.y += slideUp;   // Ridicare spre tavan
animPos.z -= slideOut;  // Iesire usoara in exterior
```

2. Sincronizarea Fizica (Optimizare) In loc sa recalculam limitele AABB (Min/-Max) la fiecare cadru si sa riscam erori de aproximare a rotatiei, am implementat o optimizare logica. Coliziunea cu usa este verificata doar daca aceasta se afla intr-o stare "periculoasa" (inchisa sau intredeschisa).

- **Stare Inchisa ($< 70°$):** Bounding box-ul static al usii este considerat activ. Daca algoritmul `checkCollision` detecteaza intersectia, masina este oprita.
- **Stare Deschisa ($> 70°$):** Se considera ca vehiculul are loc sa treaca pe sub usa. Verificarea coliziunii este dezactivata complet pentru acest obiect ("Passthrough"), economisind resurse de calcul.

3. Vizualizarea in Modul Debug Pentru a confirma vizual ca usa se ridica, in modul Debug (Tasta B) aplicam aceeasi matrice de transformare asupra cubului wireframe ca si asupra modelului 3D vizual.

```
// Constructia matricii pentru Bounding Box-ul vizual
glm::mat4 model = glm::mat4(1.0f);
model = glm::translate(model, animPos); // Pozitia interpolata
model = glm::rotate(model, glm::radians(garageDoorAngle), axisX);
model = glm::scale(model, doorDimensions);

// Desenarea liniilor rosii care urmaresc perfect usa
glUniformMatrix4fv(..., value_ptr(model));
glDrawArrays(GL_LINES, 0, 24);
```

Aceasta implementare asigura o tranzitie fluida intre starea de obstacol solid si calea de acces libera, oferind in acelasi timp feedback vizual corect utilizatorului.

3.7 Simularea Fizica a Vehiculului

Pentru a depasi limitarile unei miscari liniare simple (translatie directa), am implementat un model fizic simplificat care simuleaza fortele ce actioneaza asupra unui autovehicul: tractiunea, franarea, frecarea si momentul cinetic al rotilor.

Logica este centralizata in functia `processMovement` si se bazeaza pe integrarea vitezei in timp.

3.7.1 Modelul de Acceleratie si Inertie

Vehiculul nu raspunde instantaneu la comenzi. Am introdus variabila globala `currentCarSpeed` care actioneaza ca un acumulator de impuls.

Sistemul functioneaza ca o masina de stari:

- **Accelerare (Tractiune):** Cand tasta **W** este apasata, viteza creste cu o rata constanta (`acceleration`).
- **Franare (Braking):** Daca vehiculul se misca in fata si se apasa **S**, se aplica o forta de franare (`breakPower`) mult mai mare decat acceleratia, pentru oprire rapida.
- **Coasting (Inertie/Frecare):** Cand nicio tasta nu este apasata, viteza scade treptat catre zero folosind un factor de `deceleration`. Acest lucru simuleaza frecarea cu aerul si cu asfaltul.

```
// Logica de Inertie si Frecare (Cand nu se apasa taste)
if (!pressedKeys[GLFW_KEY_W] && !pressedKeys[GLFW_KEY_S]) {
    if (currentCarSpeed > 0.1f)
        currentCarSpeed -= deceleration * deltaTime; // Scade
           viteza
    else if (currentCarSpeed < -0.1f)
        currentCarSpeed += deceleration * deltaTime; // Creste
           spre 0
    else
        currentCarSpeed = 0.0f; // Oprire completa
}
```

3.7.2 Calculul Traiectoriei si Directia

Pozitia masinii pe harta este actualizata folosind trigonometrie, pe baza unghiului de orientare (`carAngle`) si a vitezei curente.

Un aspect crucial pentru realism este conditionarea virajului de viteza. In realitate, o masina nu isi poate schimba vectorul de directie daca sta pe loc.

$$\Delta Angle = SteeringInput \times TurnSpeed \times \frac{CurrentSpeed}{MaxSpeed}$$

```
// Calculul noii pozitii
float moveStep = currentCarSpeed * deltaTime;
glm::vec3 nextPos = carPosition;
nextPos.x += sin(glm::radians(carAngle)) * moveStep;
nextPos.z += cos(glm::radians(carAngle)) * moveStep;
```

```

// Rotirea masinii (Heading) conditionata de viteza
float turnFactor = (currentCarSpeed / maxSpeed);
carAngle += steeringAngle * turnSpeed * turnFactor;
```

3.7.3 Cinematica Rotilor (Spin si Viraj)

Pentru un aspect vizual convingator, rotile trebuie sa se comporte independent de sasiu. Am implementat doua tipuri de transformari pentru roti:

1. **Virajul (Steering):** Rotile din fata se rotesc pe axa Y cu unghiul `steeringAngle`. Am implementat si un mecanism de "Auto-Return" care readuce rotile la centru cand tastele A/D sunt eliberate, simuland geometria directiei auto.

2. **Invartirea (Spin):** Toate cele 4 roti se rotesc pe axa lor (X local) proportional cu distanta parcursa.

```
// Calculul rotatiei vizuale a rotilor
// wheelRotationFactor determina cate grade se roteste roata per
   metru parcurs
float wheelRotationFactor = 25.0f;

if (currentCarSpeed > 0)
    wheelAngle += currentCarSpeed * wheelRotationFactor *
       deltaTime;
else
    wheelAngle += currentCarSpeed * wheelRotationFactor *
       deltaTime;
```

Aceasta valoare (`wheelAngle`) este apoi trimisa in `renderCar` si aplicata matricei de model a fiecarei roti inainte de desenare.

4 Manual de utilizare

Aplicatia este controlata folosind tastatura si mouse-ul. Mai jos sunt listate comenzile disponibile:

4.1 Control General

- **W / S**: Accelerare (fata) / Franare sau Marsarier (spate).
- **A / D**: Viraj stanga / dreapta.
- **Mouse**: Rotirea camerei in jurul masinii (in mod Car) sau privire libera (in mod Free).

4.2 Taste Functionale

- **P**: Comuta intre modul "Condus" (Third Person) si modul "Camera Libera".
- **O**: Deschide / Inchide usa garajului.
- **B**: Activeaza / Dezactiveaza afisarea vizuala a coliziunilor (Bounding Boxes - linii rosii).
- **M**: Porneste o animatie automata de prezentare a scenei (tur cinematic).

4.3 Moduri de Vizualizare (Debug)

- **Tasta 1**: Mod Wireframe (doar liniile poligoanelor).
- **Tasta 2**: Mod Poligonal/Puncte (doar varfurile).
- **Tasta 3**: Mod Solid (randare normala, implicita).

5 Concluzii si dezvoltari ulterioare

Proiectul a reusit sa indeplineasca obiectivele propuse, implementand o simulare functionala cu elemente de fizica si realiste. Implementarea coliziunilor manuale si a inertiei masinii adauga un grad ridicat de interactivitate, iar sistemul de umbre si lumini multiple contribuie la realismul scenei.

Dezvoltari ulterioare posibile:

- Implementarea umbrelor pentru sursele de lumina punctiforme (Point Shadows) folosind Cube Maps de adancime, pentru ca felinarele sa lase umbre noaptea.
- Adaugarea reflexiilor dinamice pe caroseria masinii (Environment Mapping) pentru un aspect metalic mai realist.
- Implementarea efectelor meteorologice (ceata, ploaie) pentru a masca marginile hartii.

6 Referinte

1. Documentatia oficiala OpenGL -`https://www.opengl.org/`
2. Laboratoarele de PG (UTCN).
3. LearnOpenGL - `https://learnopengl.com/` (Tutoriale pentru Bounding Boxuri, Particule Fum,Lumini etc).
4. Modele 3D (TurboSquid) - https://www.turbosquid.com/
5. Modele 3D (Free3D) - https://free3d.com/